Production Deployment SOP (Manual Traditional VPS Model)

1. PURPOSE

This document defines the official production deployment procedure for the project running on Ubuntu Server using FastAPI, Gunicorn, Nginx, MariaDB, systemd, and manual Git-based deployment. The objective is to ensure controlled, repeatable, and secure deployments without automation pipelines.

2. SCOPE

This SOP applies to backend deployment, frontend build deployment, database migrations, service restarts, rollback handling, and production monitoring on the live VPS environment.

3. ENVIRONMENT ARCHITECTURE

Operating System: Ubuntu Server. Backend: FastAPI running through Gunicorn. Process Manager: systemd. Reverse Proxy: Nginx. Database: MariaDB (local instance). Frontend: React production build served via Nginx. SSL: Certbot (Let's Encrypt).

Traffic Flow: User → Nginx (HTTPS) → Gunicorn (Unix Socket) → FastAPI → MariaDB.

4. ACCESS CONTROL

Only authorized personnel may SSH into production. Root login is disabled. Access must be key-based. All production changes must be approved prior to deployment. No direct modification of files outside the defined deployment steps.

5. PRE-DEPLOYMENT CHECKLIST

Before any production deployment: Code must be reviewed and merged into main branch. Local tests must pass. Database migration scripts must be validated. Production database

backup must be created using mysqldump. Change impact must be evaluated and approved.

6. MANUAL DEPLOYMENT PROCEDURE

6.1 SSH Access

SSH into the production server using authorized credentials. Confirm correct environment.

6.2 Code Update

Navigate to project directory. Execute: git pull origin main. Verify no merge conflicts.

6.3 Virtual Environment

Activate virtual environment. Install or update dependencies using pip install -r requirements.txt.

6.4 Database Migration

Run database migration scripts. Confirm schema integrity and validate no errors occurred.

6.5 Frontend Build

Navigate to frontend directory. Run npm install if required. Execute npm run build for production assets.

6.6 Service Restart

Restart backend service using: systemctl restart tta. Verify service status with systemctl status tta.

6.7 Log Verification

Monitor logs using journalctl -u tta -f. Verify Nginx configuration using nginx -t. Confirm frontend and API endpoints respond correctly.

7. ROLLBACK PROCEDURE

If deployment fails: Checkout last stable commit using git checkout . Reinstall dependencies if required. Restore database from latest backup if schema affected. Restart service and verify logs.

8. BACKUP POLICY

Daily database backup must be executed using mysqldump via cron job. Backups must be stored off-server securely. Backup restoration should be tested periodically.

9. MONITORING AND HEALTH CHECK

Daily health check must include verification of CPU usage, memory usage, disk space, service status, database connectivity, and SSL certificate validity. Any anomaly must be recorded and escalated.

10. INCIDENT CLASSIFICATION

P0: Application down – immediate rollback. P1: Core feature failure – urgent fix. P2: Minor defect – scheduled patch.

11. VERSION CONTROL AND DOCUMENT MAINTENANCE

All deployments must reference a specific Git commit or tag. This SOP must be versioned and updated upon any architectural change.